

Neural Networks

Faculty of Information Technology
Monash University, Australia

Week 5

In-Class Quiz

- The quiz must be attempted in person during the Week 6 seminar. Any attempt outside of this seminar will be reported as a breach of academic integrity.
- The quiz covers content from Weeks 1 to 5.
- The quiz will run from **2:50 pm to 3:50 pm** on Wednesday, **3 September**.
- Bring your student ID card (physical card or digital M-Pass) or another form of photo ID for authentication purposes.
- This is an open-book quiz: you may bring and consult any notes, either paper-based or as digital PDF files.

In-Class Quiz



- You may use a calculator.
- You are NOT allowed to use any other software (e.g. Python or R), any online services or platforms (e.g. Google, ChatGPT, Claude), or any communication tools (e.g. email, SMS, WhatsApp, WeChat) during the quiz.
- Blank paper may be brought for rough work and calculations, but it will not be collected or marked.
- All answers must be entered in Moodle.

Outline



- Multilayer Neural Networks
- Convolutional Neural Networks
- Attention and Transformers
- Large Language Models

Learning Outcomes

- After learning this module, you should be able to:
- Understand single layer and multilayer neural networks.
 - Understand the key components of convolutional neural networks.
 - Understand attention and the transformer architecture.
 - Utilize commonly used neural network models to solve unsupervised and supervised real world tasks.

History of Deep Learning

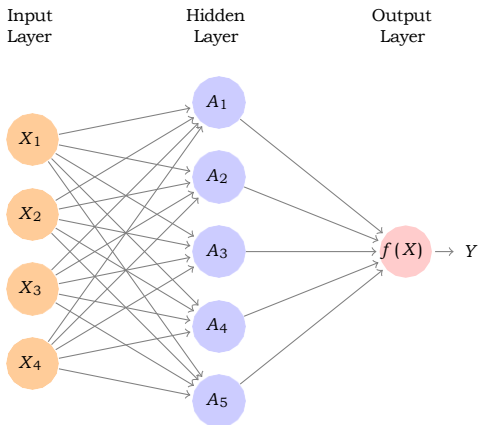
- 1980s: Neural networks become popular.
- 1990s–2000s: SVMs, boosting and other methods dominate.
- Since 2006: Neural networks re-emerge as *Deep Learning*.
- Vast improvements in computing power, larger training sets, and software: Tensorflow and PyTorch.
- Yann LeCun, Geoffrey Hinton and Yoshua Bengio (2019 ACM Turing Award).
- Geoffrey Hinton: 2024 Nobel Prize in Physics, jointly with John Hopfield.



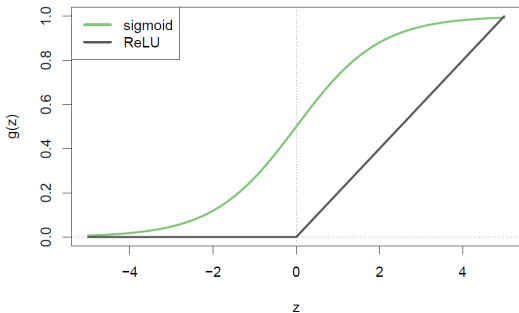
Single Hidden Layer Neural Networks



$$\begin{aligned} \triangleright f(\mathbf{X}) &= \beta_0 + \sum_{k=1}^K \beta_k h_k(\mathbf{X}) \\ &= \beta_0 + \sum_{k=1}^K \beta_k g(w_{k0} + \sum_{j=1}^p w_{kj} X_j). \end{aligned}$$

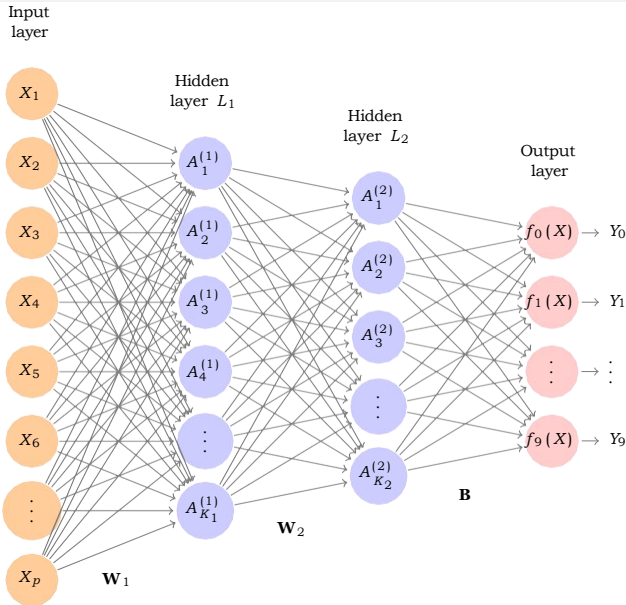


Activation Function



- $h_k(\mathbf{X}) = g(w_{k0} + \sum_{j=1}^p w_{kj}X_j)$ is called the *activation function*, such as *sigmoid* and *rectified linear*.
- Activation functions in hidden layers are typically nonlinear, otherwise the model collapses to a linear model.

Multi-Layer Perceptron



The Output Layer

➤ Let $Z_m = \beta_{m0} + \sum_{l=1}^{K_2} \beta_{ml} A_j^{(2)}$,

➤ Regression:

$$f_m(X) = Z_m$$

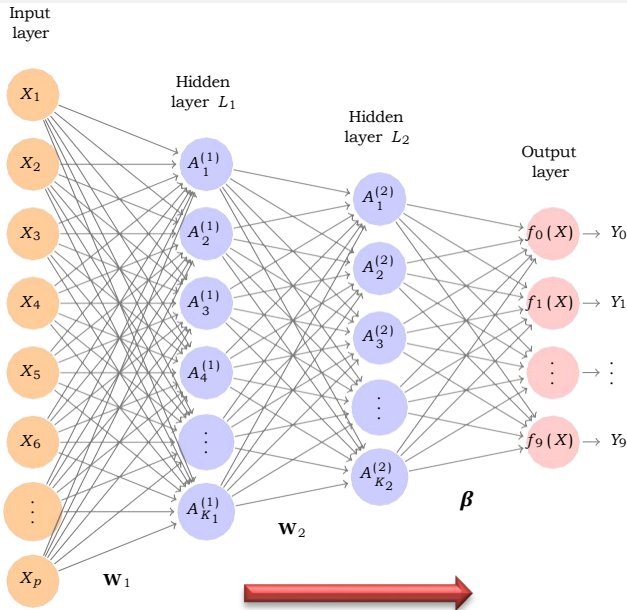
➤ Binary classification:

$$f_m(X) = \Pr(Y = 1|X) = \frac{1}{1 + e^{-Z_m}}$$

➤ Multi-class classification:

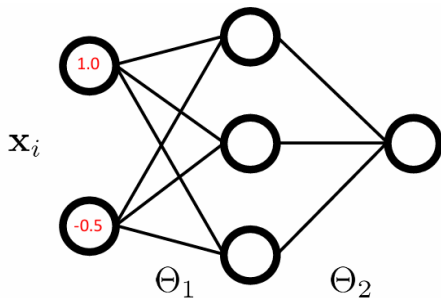
$$f_m(X) = \Pr(Y = m|X) = \frac{e^{Z_m}}{\sum_{l=1}^K e^{Z_l}}$$

Forward Propagation



Forward Propagation (Example)

$$\mathbf{x}_i = \begin{pmatrix} 1.0 \\ -0.5 \end{pmatrix} \quad \Theta_1 = \begin{pmatrix} 1.2 & 2.1 & 1.5 \\ -0.3 & -0.7 & 0.3 \end{pmatrix} \quad \Theta_2 = \begin{pmatrix} -0.2 \\ 0.5 \\ 1.3 \end{pmatrix}$$

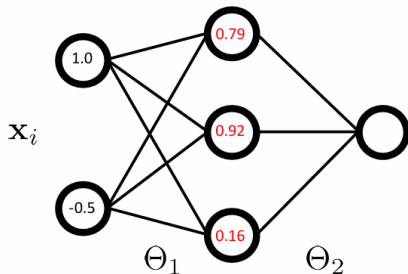


Forward Propagation (Example)

$$\mathbf{x}_i = \begin{pmatrix} 1.0 \\ -0.5 \end{pmatrix} \quad \Theta_1 = \begin{pmatrix} 1.2 & 2.1 & 1.5 \\ -0.3 & -0.7 & 0.3 \end{pmatrix} \quad \Theta_2 = \begin{pmatrix} -0.2 \\ 0.5 \\ 1.3 \end{pmatrix}$$

$$\Theta_1^\top \mathbf{x}_i = \begin{pmatrix} 1.2 & -0.3 \\ 2.1 & -0.7 \\ -1.5 & 0.3 \end{pmatrix} \begin{pmatrix} 1.0 \\ -0.5 \end{pmatrix} = \begin{pmatrix} 1.35 \\ 2.45 \\ -1.65 \end{pmatrix}$$

$$\mathbf{h}_1 = \sigma(\Theta_1^\top \mathbf{x}_i) = \begin{pmatrix} \sigma(1.35) \\ \sigma(2.45) \\ \sigma(-1.65) \end{pmatrix} = \begin{pmatrix} 0.79 \\ 0.92 \\ 0.16 \end{pmatrix}$$



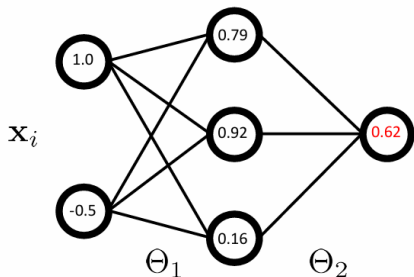
where $\sigma(x) = \frac{1}{1+e^{-x}}$

Forward Propagation (Example)

$$\mathbf{x}_i = \begin{pmatrix} 1.0 \\ -0.5 \end{pmatrix} \quad \Theta_1 = \begin{pmatrix} 1.2 & 2.1 & 1.5 \\ -0.3 & -0.7 & 0.3 \end{pmatrix} \quad \Theta_2 = \begin{pmatrix} -0.2 \\ 0.5 \\ 1.3 \end{pmatrix}$$

$$\Theta_2^\top \mathbf{h}_1 = \begin{pmatrix} -0.2 & 0.5 & 1.3 \end{pmatrix} \begin{pmatrix} 0.79 \\ 0.92 \\ 0.16 \end{pmatrix} = 0.51$$

$$\hat{y}_i = \sigma(\Theta_2^\top \mathbf{h}_1) = \sigma(0.51) = 0.62$$



Activity 1: Compute $Y = XW + b$



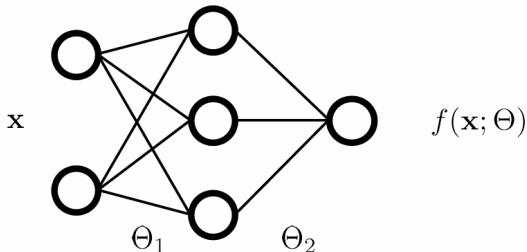
- Write the code to compute $Y = XW + b$, where
 - X is a batch of input vectors and each vector is of size 10. The batch size is 8.
 - W is the weight matrix with shape (10, 4), and
 - b is the bias vector.
- What is the shape of Y ?

Training: Overview

- Objective: Find the parameters that minimise the loss.

$$\min_{\Theta} \frac{1}{n} \sum_{i=1}^n \ell(f(\mathbf{x}_i; \Theta), y_i) := L(\Theta)$$

Loss Function



Training: Loss Functions

➤ Regression: $\sum_{i=1}^n (f_m(\mathbf{X}_i) - y_i)^2$

➤ Binary classification:

$$\sum_{i=1}^n -y_i \log(f_m(\mathbf{X}_i)) - (1 - y_i) \log(1 - f_m(\mathbf{X}_i))$$

➤ Hinge loss: $\sum_{i=1}^n \max(0, \lambda - y \cdot f_m(\mathbf{X}_i))$ for SVM.

➤ Multi-class classification: $-\sum_{i=1}^n \sum_{m=1}^M y_{im} \log(f_m(\mathbf{X}_i))$

Training: Regularization

- Regression:

$$-\sum_{i=1}^n (f_m(\mathbf{X}_i) - y_i)^2 + \lambda \|\boldsymbol{\beta}\|_2^2$$

- Multi-class classification for M classes:

$$-\sum_{i=1}^n \sum_{m=1}^M y_{im} \log(f_m(\mathbf{X}_i)) + \lambda \|\boldsymbol{\beta}\|_2^2$$

- More regularization techniques, such as dropout.

Training: Parameter Update

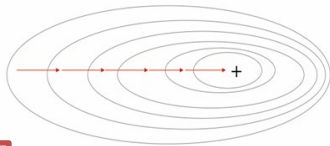
- (Stochastic) gradient descent updates parameters iteratively to the gradient direction.

$$\Theta^{(t+1)} = \Theta^{(t)} - \gamma \nabla L(\Theta^{(t)})$$

parameters

Learning rate

Derivative

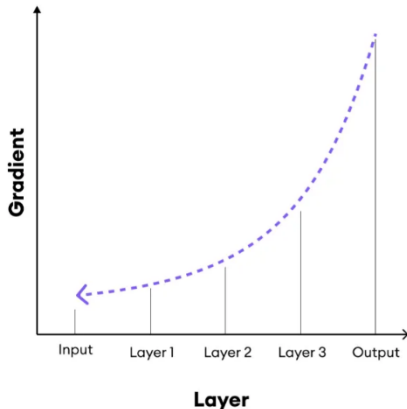


- Backpropagation
 - First adjust the last layer parameters.
 - Propagate errors back to each previous layers.
 - Adjust previous layer parameters.
 - Implement the chain-rule to compute derivatives.

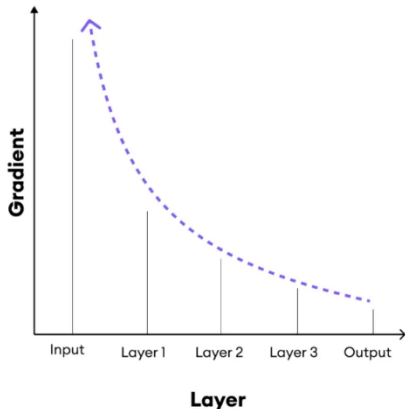
Vanishing and Exploding Gradients

During backpropagation, gradients may diminish or grow exponentially in magnitude as they traverse multiple layers.

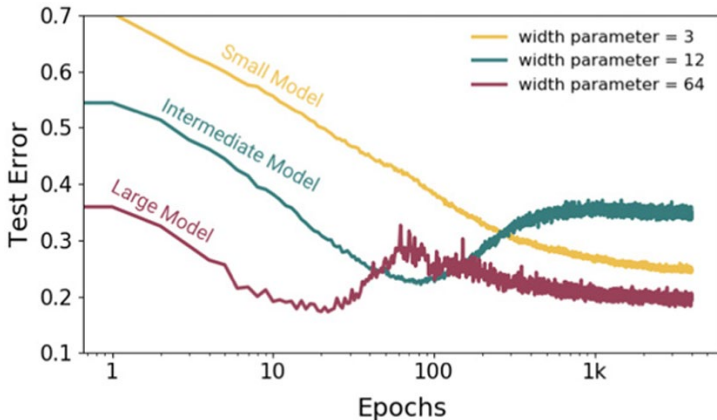
Vanishing Gradient



Exploding Gradient

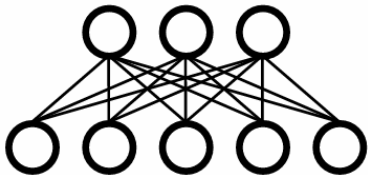


Training: Double-Descent



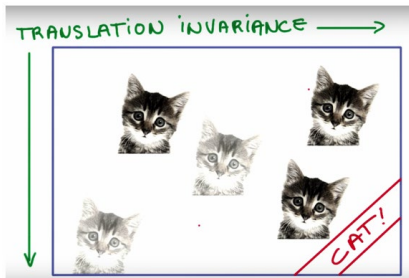
Drawbacks of MLP

- Fully connected layers



- Drawbacks:

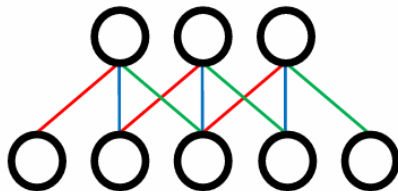
- Easy to be over-fitted.
- Difficult to train a deep MLP.
- Do not enforce any structure.
- Difficult to achieve translation invariance.
- Vanishing and exploding gradient problems.



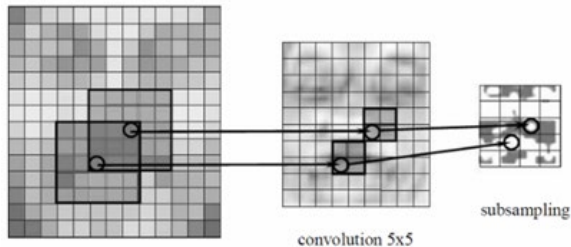
Convolutional Neural Networks



- Convolution: Parameter sharing and local connectivity



- Pooling.



CNN: Convolution

- Apply the same parameters over different spatial regions.

1	0	1
0	1	0
1	0	1

Filter / Kernel

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image

CNN: Convolution

- Apply same parameters over different spatial regions.

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

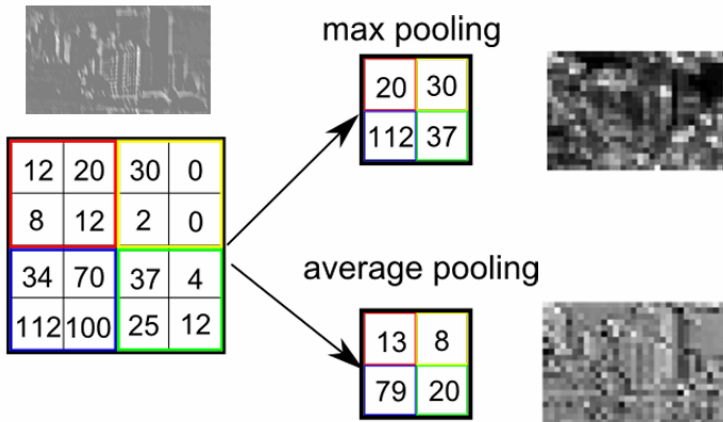


4		

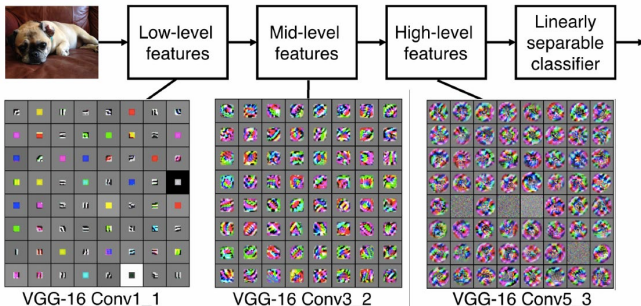
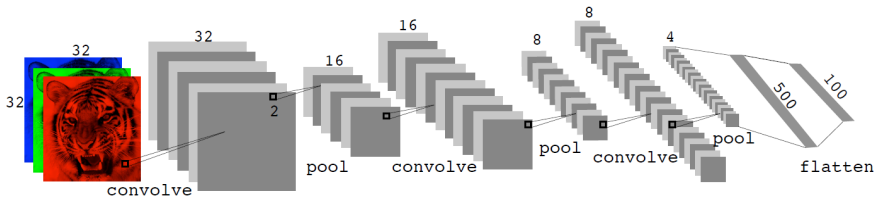
Convolved Features

CNN: Pooling

- Enhance translation invariance.
- Regularization effect.



Architecture of CNN



Representing Text: Tokenisation

- Neural networks operate on vectors, so text is first split into discrete units called tokens.
 - Word-level vocabularies are huge and cannot represent unseen words.
 - Modern LLMs use subword tokenisation such as Byte Pair Encoding (BPE).
 - Frequent words remain whole; rare words are split into subwords.
 - Each token ID is mapped to a learned embedding vector.

Example: "the film was unbelievable"

Word-level



4 tokens • huge vocabulary • rare word → out-of-vocabulary <UNK>

BPE

(subword)



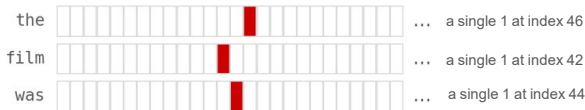
6 tokens • small vocabulary (~50K) • no <UNK>: unseen words are handled

Word Embeddings

Same BPE tokens:



One-hot (sparse, $|V| \approx 50,000$ dimensions)



all vectors orthogonal: no notion of similarity, huge and wasteful

Embedding (dense, learned, $d \approx 768$)



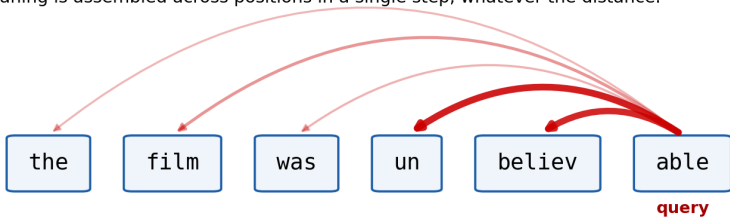
similar tokens map to nearby vectors;
learned jointly with the model

Each token ID is mapped to a learned embedding vector.

Why Attention?

- Language is a variable-length sequence, and token meaning depends on context.
 - MLPs need fixed-size inputs; CNNs only mix information within a local window.
 - In the BPE example, “un”, “believ” and “able” mean little alone; “un” flips the meaning of “believ + able”.
 - Attention lets every position access every other position in a single step, with learned weights.

Attention weights of the query “able”: strongest on “un” and “believ”, so the word’s meaning is assembled across positions in a single step, whatever the distance.



Attention

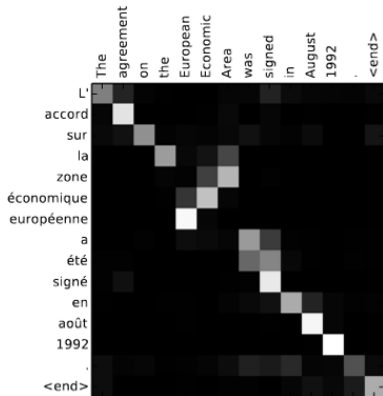
$$A = \sum_{t=1}^T \text{softmax}(f(Q, k_t)) \cdot v_t$$

Query

Key

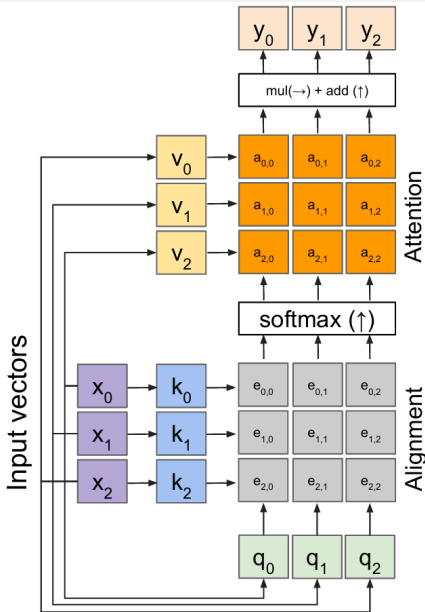
Value

Where $\text{softmax}(\mathbf{z}) = [\frac{e^{z_1}}{\sum_{j=1}^T e^{z_j}}, \dots, \frac{e^{z_T}}{\sum_{j=1}^T e^{z_j}}]$



Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate.

Self Attention



Outputs:

context vectors: $\mathbf{y}$ (shape: D_v)

Operations:

Key vectors: $\mathbf{k} = \mathbf{x} \mathbf{W}_k$

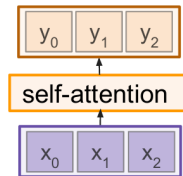
Value vectors: $\mathbf{v} = \mathbf{x} \mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x} \mathbf{W}_q$

Alignment: $e_{i,j} = \mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

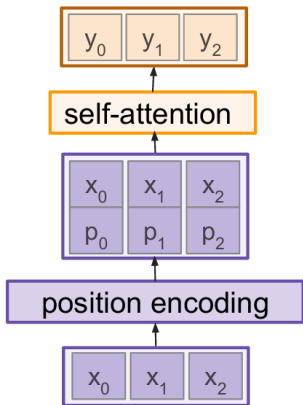
Output: $y_j = \sum_i a_{i,j} v_i$



Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Position Encoding



Use a function to map each position j to a d -dimensional vector.

Intuition

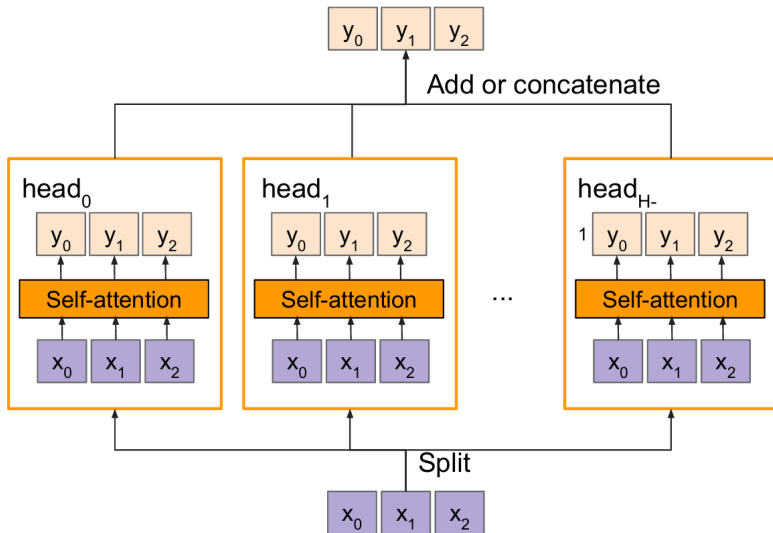
0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

$$p(t) = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \\ \vdots \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_d$$

Concatenate special positional encoding p_j to each input vector x_j .

$$\text{where } \omega_k = \frac{1}{10000^{2k/d}}$$

Multi-Head Self Attention



Transformers

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

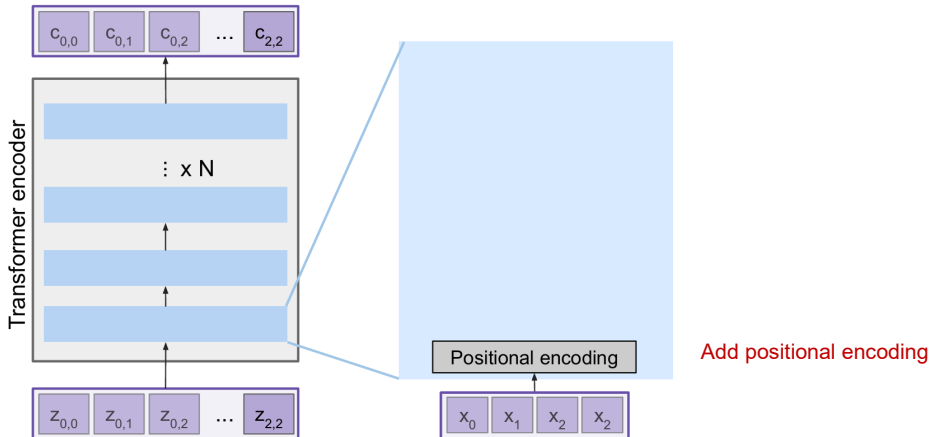
Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

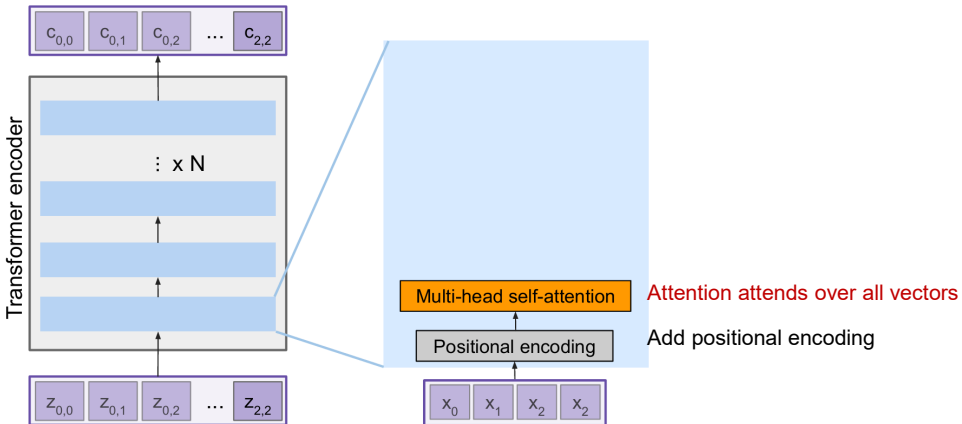
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

265803 citations!

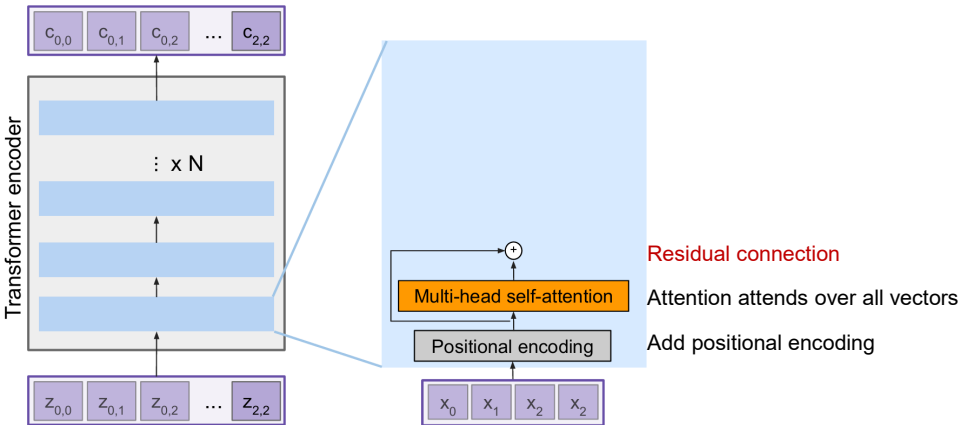
Transformer Encoder Block



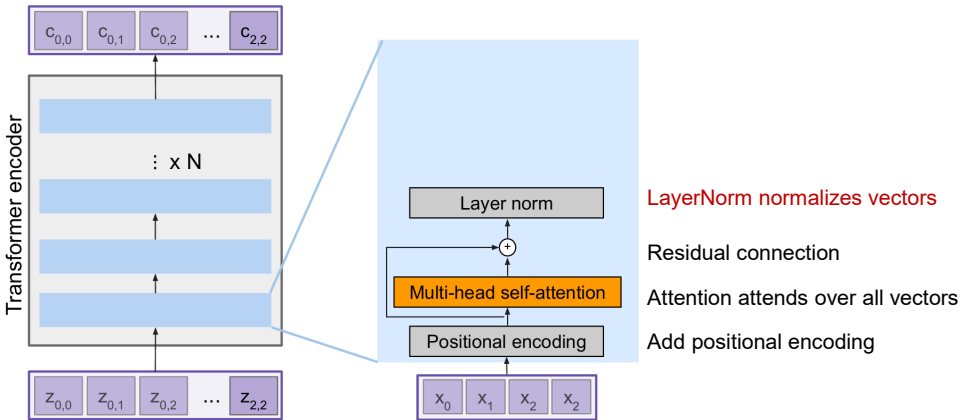
Transformer Encoder Block



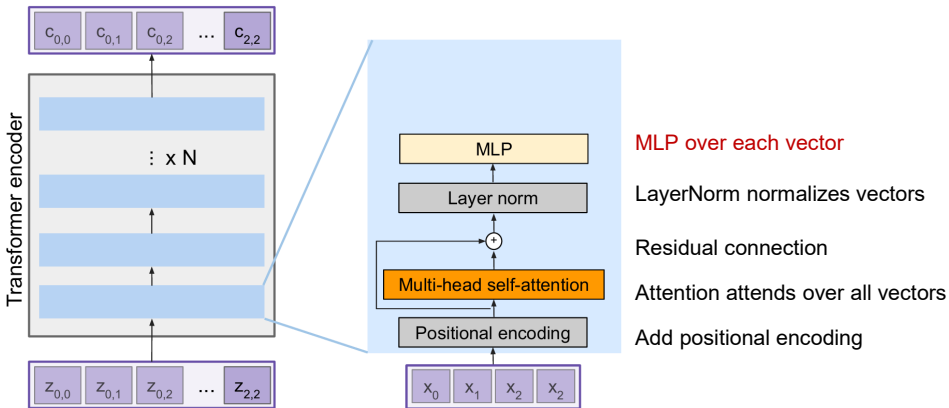
Transformer Encoder Block



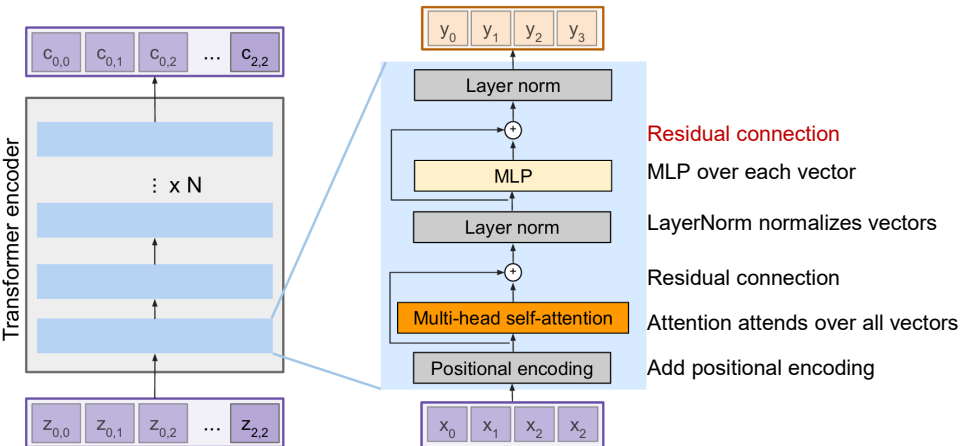
Transformer Encoder Block



Transformer Encoder Block



Transformer Encoder Block

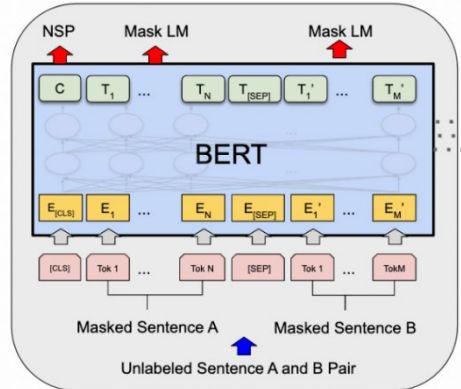


BERT

x : [CLS] Joe Biden is the US president. [SEP] He was inaugurated on January...

$\tilde{x}$: [CLS] Joe <mask> is the US <mask>. [SEP] He <mask> inaugurated on January...

1. Create a masked sentence for each input sentence.
2. Train BERT to predict the original words from context.
3. Train BERT to predict whether B naturally follows A or a random choice.



Language Modelling: Next-Token Prediction

- A language model assigns a probability to a sequence of tokens.
 - Objective: predict the next token from all previous tokens.
 - Self-supervised: the training signal comes from raw text, with no manual labels.
 - The loss is cross-entropy over the vocabulary at every position.
 - At web scale, this objective forces the model to learn syntax, world knowledge and some reasoning patterns.

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, \dots, x_{t-1})$$

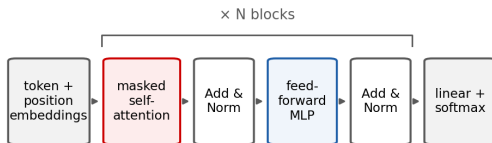
$$P(\text{the, film, was, un, believ, able}) = P(\text{the}) \cdot P(\text{film} | \text{the}) \cdot P(\text{was} | \text{the, film}) \\ \dots P(\text{able} | \text{the, film, was, un, believ})$$

training maximises the log-probability of every next token in the corpus

Decoder-Only Transformers: GPT



- GPT-style models stack transformer blocks trained by next-token prediction.
 - Causal (masked) self-attention: each token attends only to earlier positions.
 - Training is parallel across positions; generation is autoregressive, one token at a time.
 - Any text task can be framed as text continuation.
 - Examples: the GPT series, Llama and most current chat models.

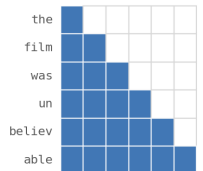


input: the film was un believ → next token: **able**

training: all positions predicted in parallel

generation: autoregressive, one token at a time

causal mask



each row attends to itself and earlier positions only

Pre-Training and Fine-Tuning

- Modern NLP follows a two-stage paradigm.
 - Pre-training: self-supervised learning on massive unlabelled corpora (next-token prediction for GPT, masked-token prediction for BERT).
 - Fine-tuning: continue training on a smaller labelled dataset for the target task.
 - Pre-trained representations transfer across tasks and cut the labelled data required.
 - Parameter-efficient methods such as LoRA fine-tune only a small fraction of the weights.

1. Pre-training (self-supervised)

billions of web-text tokens, no labels:
the film was un believ → ?
target next token: able

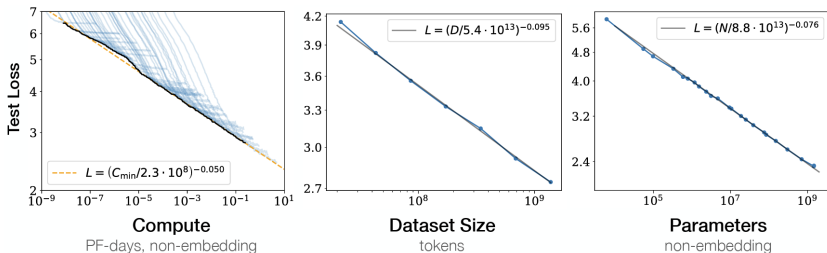
transfer
→

2. Fine-tuning (supervised, small)

e.g. sentiment labels for movie reviews:
“the film was unbelievable”
label: Positive

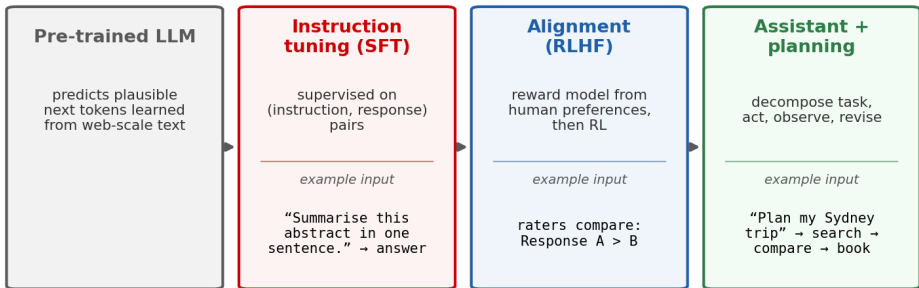
Scaling Language Models

- LLM performance improves predictably with model size, data and compute.
 - Scaling laws: test loss falls smoothly as a power law in parameters, data and compute (Kaplan et al., 2020).
 - Compute-optimal training balances model size against the number of training tokens (Hoffmann et al., 2022).
 - Scale, data quality and post-training together drive recent capability gains.



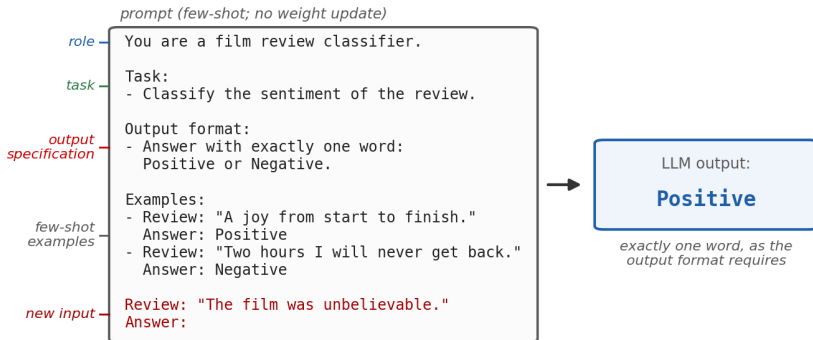
From LLMs to Assistants

- A pre-trained LLM predicts plausible text; alignment turns it into an assistant.
- Instruction tuning: supervised fine-tuning on instruction-response pairs.
 - RLHF: reward model from human preferences, then optimise with RL.
 - Planning: decompose a task into steps, act, observe and revise.
 - Examples: ChatGPT, Claude and Gemini are aligned pre-trained LLMs.



In-Context Learning

- LLMs can perform new tasks from the prompt alone, without any weight update.
 - Zero-shot: describe the task in natural language.
 - Few-shot: include a handful of input-output examples in the prompt.



Chain-of-Thought

- Chain-of-thought (CoT) prompting elicits intermediate reasoning steps before the answer.
 - Few-shot exemplars with worked reasoning, or simply “Let’s think step by step”, change how the model answers.
 - Large gains on arithmetic, symbolic and multi-step reasoning tasks, emerging with sufficient model scale.

Standard prompting

input (prompt)

Q: A cinema has 23 rows of 18 seats.
350 tickets are sold. How many
seats stay empty?
Give the answer.



model output

44 X
(guessed in one step)

the model maps question $\rightarrow$ answer directly;
multi-step arithmetic often fails

Chain-of-thought prompting

input (prompt)

Q: A cinema has 23 rows of 18 seats.
350 tickets are sold. How many
seats stay empty?
Let’s think step by step.



model output

Total seats: $23 \times 18 = 414$.
Empty: $414 - 350 = 64$.
The answer is 64. ✓

the instruction is part of the input; the
model generates the steps, then the answer

Summary

- What we learned:
 - The high performance of DNNs benefits from deep structure.
 - CNNs capture local spatial structures and enable translational invariance.
 - Self-attention lets every token attend to every other token in a sequence.
 - Transformers stack self-attention, residual connections and layer normalisation, largely avoiding vanishing gradients.
 - LLMs are transformers pre-trained on next-token or masked-token prediction at scale, then fine-tuned and aligned.
- Relevant readings:
 - Chapter 10 of “An Introduction to Statistical Learning”; Vaswani et al. (2017) “Attention Is All You Need”.
- Acknowledgement
 - Some figures are from “An Introduction to Statistical Learning with Applications in Python/R” and the courses cs231n and cs330.

References (1/3)

- Bahdanau, D., Cho, K. and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. ICLR.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017). Attention is all you need. NeurIPS.
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019). BERT: pre-training of deep bidirectional transformers for language understanding. NAACL.
- Sennrich, R., Haddow, B. and Birch, A. (2016). Neural machine translation of rare words with subword units. ACL.
- Radford, A., Narasimhan, K., Salimans, T. and Sutskever, I. (2018). Improving language understanding by generative pre-training. OpenAI.
- Brown, T. et al. (2020). Language models are few-shot learners. NeurIPS.
- Nakkiran, P., Kaplun, G., Bansal, Y., Yang, T., Barak, B. and Sutskever, I. (2021). Deep double descent: where bigger models and more data hurt. Journal of Statistical Mechanics.

References (2/3)

- Kaplan, J. et al. (2020). Scaling laws for neural language models. arXiv:2001.08361.
- Hoffmann, J. et al. (2022). Training compute-optimal large language models. NeurIPS.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W. (2022). LoRA: low-rank adaptation of large language models. ICLR.
- Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. NeurIPS.
- Wei, J. et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. NeurIPS.
- James, G., Witten, D., Hastie, T. and Tibshirani, R. An Introduction to Statistical Learning with Applications in Python/R, Chapter 10.
- Course materials: Stanford CS231n (Convolutional Neural Networks for Visual Recognition) and CS330 lecture notes, figure sources as acknowledged.

References (3/3)



- Bengio, Y., Ducharme, R., Vincent, P. and Jauvin, C. (2003). A neural probabilistic language model. Journal of Machine Learning Research.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. and Dean, J. (2013). Distributed representations of words and phrases and their compositionality. NeurIPS.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y. and Iwasawa, Y. (2022). Large language models are zero-shot reasoners. NeurIPS.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. and Cao, Y. (2023). ReAct: synergizing reasoning and acting in language models. ICLR.